

MID-STATE TECHNICAL COLLEGE • IT PROGRAMS

Application Scanning

Units 4 & 5 • Ethical Hacking (10-151-160)



Learning Outcomes

By the end of these two units, you will be able to:



Explain how a web application works

Describe the HTTP request and response cycle and read status codes as evidence.



Identify web ports and services

Recognize the ports applications and APIs use, and why a port suggests rather than proves.



Intercept and modify traffic

Capture a request in a proxy, alter it after client-side validation, and forward it.



Enumerate an application

Spider, brute-force unlinked content, fingerprint the platform, and fuzz an input.



Assess the defensive position

Identify the log and WAF evidence you leave, and map findings to the OWASP Top 10.

Scope and Authorization

Application scanning is far more intrusive than the port scanning you did in Unit 3.

- Authorized targets are `www.mstclab.com`, a vulnerable app you installed locally, or a box you joined on **Hack The Box** or **TryHackMe**.
- A fuzzer does not just look - it **creates accounts, submits forms, deletes records**, and can take a fragile site offline.
- Unauthorized testing is a crime under the **Computer Fraud and Abuse Act** and **Wis. Stat. §943.70** whether or not anything breaks.
- Everything you produce this unit is **TLP:AMBER** - classroom and instructor only.

Scope is confirmed in writing before any tool runs — including the tools you only meant to point at yourself.



SECURITY FOCUS

If you are not certain a target is in scope, ask before you send the request. "I thought it was allowed" is not a defense.

Why it matters: Intent is not an element of the offense. The request itself is the act.



LESSON 1

How the Web Actually Works

What Is a Web Site or Application?

Strip away the visual layer and a web application is remarkably plain.

- The **request** is a method, a path, an HTTP version, headers, and optionally a body - **entirely under the client's control**.
- The **response** is a status line, headers, and a body of HTML, JSON, XML, or binary content.
- The browser **renders; it does not decide**. Anything it enforces can be bypassed by not using a browser.
- HTTP is **stateless** - sessions exist only because the client keeps presenting a cookie or token.



Source vs. inspect

View source shows what the **server sent**. Inspect element shows what **JavaScript has since built**. The difference between the two is frequently the interesting part.

Anatomy of a Request and Response

The fastest way to see what a server actually says is to stop using a browser:

```
$ curl -v https://www.mstclab.com
> GET / HTTP/1.1          < HTTP/1.1 200 OK
> Host: www.mstclab.com   < Server: nginx
> User-Agent: curl/8.5.0  < Content-Type: text/html; charset=UTF-8
```

- `curl -v` prints the full request and response including every header.
- `curl -I` sends a **HEAD** request - headers only, no body. Fast existence check.
- Browser dev tools (**F12**) show every request a page makes, including the ones it makes on your behalf.

LIVE DEMO

Run `curl -v` against the lab target and read the exchange line by line.

Reading Status Codes

The status code is the server telling you what it did. It is the primary signal you work from.

Class	Meaning	You will meet	What it tells a scanner
1xx	Informational	101 Switching Protocols	Rare; a WebSocket upgrade
2xx	Success	200 OK, 201, 204	A 200 on a guessed path is the hit
3xx	Redirection	301, 302, 304	302 to login = exists and protected
4xx	Client error	400, 401, 403, 404, 429	403 confirms the path is real
5xx	Server error	500, 502, 503	Your input reached code that broke

Table bottom is 5.05. The difference between 403 and 404 is the most under-used signal in content discovery.

The Soft 404 Problem

Plenty of applications return a friendly "page not found" page with a 200 status.

- A discovery tool filtering only on **status code** will report **every single guess** as a hit.
- Filter on **response size**, **word count**, or **line count** instead - real pages differ in length from the error page.
- In `ffuf` that is `-fs`, `-fw`, and `-fl`. Establish the baseline first by requesting a path you know does not exist.



Build the baseline

Request `/zzz-does-not-exist` first. Whatever comes back is the shape of "nothing here" on this application - filter it out, then scan.



LESSON 2

Ports, Languages, and the Split

Ports Web Applications Use

Web traffic clusters on a handful of ports, but nothing enforces that.

Port	Service	Why you care
80 / 443	HTTP / HTTPS	Still on 80 with no redirect is worth a note
8080, 8000, 8888	Alternate HTTP	Dev servers and app servers behind a proxy
8443	Alternate HTTPS	Appliance and management consoles
3000, 5000, 5173	Framework defaults	A dev service that escaped into production
10000, 2082-2083	Webmin, cPanel	Control panels administer the whole host

A port can be used by anything: SSH runs happily on 443, and malware picks 443 because nobody blocks it. Run `nmap -sV`; never trust the SERVICE column.

Web Programming Languages

Recognition, not fluency - the language tells you what class of flaw to look for first.

Runs in the browser

- **HTML / HTML5** - hidden fields, commented-out credentials
- **CSS / CSS3** - rarely a flaw, frequently a fingerprint
- **JavaScript** - the bundled file is often the richest source of endpoints, keys, and internal hostnames on the site

Runs on the server

- **PHP** - WordPress and much of the web; file inclusion and upload flaws
- **Python** - Django, Flask; template injection, debug pages left on
- **Java** - deserialization, exposed management consoles
- **Node.js** - enormous dependency trees; **Ruby** - mass assignment

Read the JavaScript bundle on every engagement. Developers ship far more in it than they realize.

Server Side vs. Client Side

Server side

- Code executes on the server and returns data
- Example: Python querying a SQL database
- You cannot read it - you only see its output
- You influence it entirely through what you send
- **Injection, authorization, and business logic flaws live here**

Client side

- Code executes locally in your browser
- Example: JavaScript rejecting a weak password
- You can read all of it, and change or skip all of it
- A dropdown with three options is still a text field on the wire
- **Cross-site scripting and leaked secrets live here**

The premise of this unit: The client is not trustworthy, because the client is the attacker.

Client-Side Validation Is Not a Control

A check that runs before the request leaves your machine is a convenience, not a control.

- Let the JavaScript **accept** a strong password, then replace it with a weak one in the intercepted request.
- Hidden fields, disabled dropdowns, price values, and quantity limits are **all just text** once the request is on the wire.
- A file upload that "only accepts .jpg" is enforcing that in the browser - not necessarily on the server.



SECURITY FOCUS

Every validation performed in the browser must be performed again on the server. The browser version exists so users get fast feedback.

Why it matters: The server-side check is the only one an attacker cannot skip.

LESSON 3

Requests, Methods, and APIs



GET and POST

GET – parameters in the URL

- `/form.php?var1=val1&var2=val2`
- Visible in the address bar, history, and bookmarks
- Logged by the web server, proxies, CDNs, and sent in the Referer header
- Should be safe and idempotent - a GET that deletes is a design flaw

POST – parameters in the body

- `POST /form.php HTTP/1.1`
- `Host: foobar.com`
- `var1=val1&var2=val2`
- Used for logins, purchases, uploads - anything that writes

POST is not "more secure" than GET: Both are plain text; only TLS protects either. POST just keeps data out of URLs, logs, and referrer headers.

The Other HTTP Methods

GET and POST are two of eight. The rest are where authorization gets forgotten.

Method	What it does	Why it matters to you
HEAD	Returns headers only	Cheap existence check, no body downloaded
PUT / PATCH	Create or modify a resource	Occasionally left enabled on a web server
DELETE	Removes a resource	Test carefully - it does what it says
OPTIONS	Lists allowed methods	Sometimes answers honestly and saves you guessing
TRACE	Echoes the request back	No legitimate production use; should be disabled

Method matters to authorization: An application that checks permissions on POST but not on PUT is a broken access control finding. Test every method, every time.

APIs and Endpoints

An API is the same HTTP exchange with the presentation layer removed.

- An **endpoint** is a path that accepts requests - `/api/v1/users/42`.
- REST conventions put intent in the method: `GET` reads, `POST` creates, `PUT` updates, `DELETE` removes.
- JSON responses frequently carry **more fields than the page ever displays**. Read the raw response, not the rendered page.
- Check `/swagger`, `/openapi.json`, `/graphql` - documentation endpoints sometimes hand you the whole map.



SECURITY FOCUS

The classic API finding is **broken object level authorization**: the app checks that you are logged in, but not that record 43 belongs to you.

Why it matters: Changing `/users/42` to `/users/43` takes one second. Try it first, every time.

LIVE DEMO

Call a public API, read the raw JSON, then increment the ID in the path.

LESSON 4

Proxying and Request Manipulation



What Proxying Is

An intermediary that receives traffic on one side and passes it along on the other.

- **Forward proxy** sits in front of clients - corporate web filtering and content inspection.
- **Reverse proxy** sits in front of servers - load balancers, CDNs, and WAFs.
- **Intercepting proxy** is the testing tool: your browser sends everything through it, and it **pauses each request** for inspection.
- A proxy is where you get to **hold a request still and change it** before it reaches the server.



SECURITY FOCUS

To read HTTPS the proxy presents **its own certificate** and re-encrypts to the server. That is why you install its CA cert.

Why it matters: Never install that CA in the browser you bank in. Use a separate profile.

What a Proxy Lets You Do

Every one of these is a test you will run in the lab:

- **Bypass client-side validation** - let the JavaScript pass, then weaken the value. If it is accepted, server-side validation is missing.
- **Change what a form was "allowed" to send** - hidden fields, price values, quantity limits, user IDs.
- **Upload something other than what was asked for** - PHP code where an avatar was expected; does the server check content or just extension?
- **Replay and iterate** - resend one request hundreds of times with one value changed. This is the mechanism behind fuzzing and brute forcing.



Scope discipline: it is easy to leave intercept on, open an unrelated tab, and generate unauthorized traffic.

The Proxy Toolchain

Four options. Burp Community covers everything in this unit.

Tool	What you get free	Trade-off
Burp Suite	Proxy, Repeater, Decoder, Comparer	Intruder rate-limited; no active scanner
ZAP	Everything, including active scanning	Left OWASP in 2023; now ZAP by Checkmarx
Caido	Generous free tier; client/server split	Newer; smaller extension ecosystem
Browser dev tools	Network tab and Edit-and-Resend	No history, no replay at scale

The Kali package for ZAP is still zaproxy. Caido is now packaged in Kali as well.

Capture, Change, Forward

The core skill of this unit. Practice it until it is boring.

- **1. Configure** - point the browser at `127.0.0.1:8080` and install the CA cert, or use the bundled browser.
- **2. Browse with intercept off** - populate the site map first. You cannot modify what you have not found.
- **3. Intercept and submit** - the request stops in the proxy instead of reaching the server.
- **4. Change the value and forward** - edit the parameter, then release it.
- **5. Read the response** - a 500 means unhandled code; a 200 where you expected rejection means a missing control.
- **Tutorial** - https://youtu.be/ouDe5sJ_uC8

LIVE DEMO

Burp Suite: intercept a login, weaken the password after validation, forward it.

LESSON 5

Fingerprinting the Platform



What a CMS Is, and What Is Inside One

Software that lets people publish content without writing code - and most of the web runs on one.

- **WordPress** - roughly 41% of all websites and about 59% of sites with a detectable CMS (W3Techs, 2026).
- **Drupal** - under 1% overall, but heavily represented in government and higher education.
- **Joomla** - declining share, still widely deployed and often unmaintained.
- Behind the login sits the **database, the user table, uploaded files, CRM and payment integrations** - a CMS often contains PII, or a path to PII.



Django is not a CMS

Django is a web **framework** - the toolkit you build with. `Wagtail` and `django CMS` are the content management systems built on top of it.

CMS Vulnerabilities: The Real Numbers

Counting CVEs against the core platform understates the risk, because core is not the attack surface.

WordPress ecosystem, 2025	Figure	What it means
New vulnerabilities	11,334	Up 42% year over year
Found in plugins	91%	Themes 9%; only six in WordPress core
Median time to exploitation	5 hours	About 45% exploited within a day
No patch at disclosure	46%	"Keep everything updated" had nothing to update to

Why these platforms: Popularity and uniformity. One working exploit against a common plugin yields millions of identical targets, which makes mass automation rational.

Source: Patchstack, State of WordPress Security in 2026.

Fingerprinting Tools

Identify the platform, then check it against what is publicly known about that version.

```
whatweb -a 3 https://www.mstclab.com
wpscan --url https://www.mstclab.com --enumerate vp,u
joomscan -u https://target droopescan scan drupal -u https://target
```

- Tells you can read yourself: `/wp-content/` paths, the `generator` meta tag, theme paths in the page source, distinctive cookie names.
- A version string is a **hypothesis, not a finding** - backported patches mean a "vulnerable" version is frequently already fixed.

LIVE DEMO

whatweb -a 3 against the lab target; read the platform and server line.

LESSON 6

Discovery and Fuzzing



Spidering

Walking an application by following every link, until nothing new turns up.

- Produces a **site map**: pages, directories, parameters, and forms as a tree.
- Directory naming conventions alone frequently **give away the CMS or framework**.
- Its limitation: a spider can **only find what is linked**. Anything reachable but unlinked is invisible to it.
- Free intelligence first - `/robots.txt` and `/sitemap.xml` are published by the site itself.



Read robots.txt first

A **Disallow** entry is a list of paths someone specifically did not want indexed - a strange and useful thing to hand a stranger.

Content Discovery: Finding the Unlinked

Ask the server about paths from a wordlist and keep the ones that answer.

Tool	Status	Notes
DIRB	Historical - still in Kali	Unmaintained, single-threaded, no real recursion
Skipfish	Historical - archived by Google	Last real release predates modern targets
ffuf	Current default	Fuzzes any position; best response filtering
feroxbuster	Current	Recurses automatically; fast enough to hurt a server
gobuster	Current	Simple; also does DNS and vhost enumeration

Wordlists are the whole game: Results depend far more on the list than the tool. Start at `/usr/share/seclists/Discovery/Web-Content/`.

ffuf in Practice

One command, and every flag is a decision you should be able to justify:

```
ffuf -w /usr/share/seclists/Discovery/Web-Content/common.txt \  
-u https://www.mstclab.com/FUZZ \  
-mc 200,301,302,401,403 -fs 0 -rate 50 -o results.json
```

- `-w` the wordlist, `-u` the URL with `FUZZ` marking the position to substitute.
- `-mc` match these status codes; `-fs` filter out this response size - your soft-404 baseline.
- `-rate` caps requests per second. **Set it deliberately** - the default will hurt a production application.

Fuzzing

Injecting malformed or semi-malformed data automatically, and watching what happens.

- **Does it break?** A 500 or a hang means input reached code that did not anticipate it.
- **Does it accept input it should not?** Negative quantities, absurd lengths, another user's ID.
- **Does the error say too much?** Stack traces and SQL syntax errors are findings and a map of the internals.
- **Did behavior change subtly?** A slightly slower response is how blind injection is detected.



Where to point it

Paths and extensions, parameter names and values, headers (`Host`, `X-Forwarded-For`, cookies), and JSON bodies - adding fields the client never sends.

There is no stealthy way to fuzz: Thousands of 404s inside a minute. Throttle, stay in scope and window, and tell the client's operations team before you start.

LESSON 7

Reporting and the Defensive View



OWASP Top 10:2025

The shared vocabulary for application findings. Current version finalized January 2026.

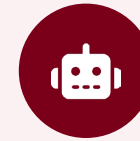
#	Category	Note
A01	Broken Access Control	Still number one; SSRF folded in here in 2025
A02	Security Misconfiguration	Much of what content discovery finds lands here
A03	Software Supply Chain Failures	New in 2025, highest debut; the plugin problem
A05	Injection	SQL, command, and cross-site scripting
A10	Mishandling of Exceptional Conditions	New in 2025; what your fuzzing tests directly

A04 Cryptographic Failures, A06 Insecure Design, A07 Authentication Failures, A08 Integrity Failures, A09 Logging and Alerting Failures complete the list. Tag every finding; never force a fit.

Bots, WAFs, and the Edge

On most real targets you are scanning through this, not at the application directly.

- **The bots** run **credential stuffing** - breached username and password pairs replayed against a login form. It works because people reuse passwords.
- **The edge** (Akamai, Cloudflare, Fastly) is a reverse proxy acting as CDN, DDoS scrubber, **WAF** (web application firewall), and bot manager at once.
- A WAF is a **compensating control, not a fix** - it buys time for patching rather than replacing it.
- Bot management uses TLS fingerprints, header ordering, and timing - not the trivially forged `User-Agent`.



What it means for you

You may be scanning **shared edge infrastructure**, not the origin. Uniform 403s make discovery useless, so findings from behind a WAF are **provisional**. Sudden 429s mean slow down.

Blue Team: What the Logs Show

Every probe is a full HTTP request the application itself records. MITRE ATT&CK T1595.003.

- **404 volume and ratio** - one source generating hundreds a minute is not a confused user.
- **Paths that never existed** - `/.env`, `/admin.bak`, `/wp-admin` on a site that never ran WordPress.
- **Sequential parameter values** - requests walking an ID upward are an access control test.
- **No static assets, no Referer** - real browsers pull the whole page; scanners ask only for what they want.
- **Default User-Agent strings** - ffuf, Nikto, and Burp still ship recognizable ones.



Mitigations

Remove backup files and `.git`. Disable directory listing and verbose errors. Validate server-side. Rate limit auth endpoints. Inventory plugins - 91% of the risk. Log **and actually alert**.

Security Focus: The Finding You Cannot Remediate

Not every finding has a fix - and pretending otherwise makes a report less useful, not more.



SECURITY FOCUS

Your scan finds an outdated plugin with a public authenticated RCE. The vendor has abandoned it, no patch exists, and the client's entire customer intake depends on it. Removing it stops the business; leaving it is a documented exposure.

Write it honestly and convert it into a control requirement: restrict the affected endpoints to authenticated internal users, add a **virtual patching rule at the WAF**, monitor the vulnerable paths specifically, set a review date, and open a replacement project with a named owner.

Why it matters: An exposure that is accepted, documented, instrumented, and deadlined is a fundamentally different risk from one nobody knew about.

Key Takeaways



A web application is text over TCP. Read the status code, not the rendered page.



The client is not trustworthy, because the client is the attacker. Validate on the server.



A spider finds what is linked; discovery finds what is not. Both need a baseline.



Core is not the attack surface - 91% of WordPress vulnerabilities were in plugins.



There is no stealthy way to fuzz. Throttle, stay in scope, and mark your findings.

MID-STATE TECHNICAL COLLEGE · IT PROGRAMS

Questions?

Units 4 & 5 · Application Scanning

